JXS Tech

Desarrollo e integración de módulos de una aplicación web

Evidencia GA8-220501096-AA1-EV01

Aprendiz: Sergio Murillo
Programa: Análisis y Desarrollo de Software
Institución: Servicio Nacional de Aprendizaje - SENA
Fecha: 4 de noviembre de 2026

Tecnologías	Verificación
React + Node.js + Express + MongoDB	18 pruebas unitarias y 38 de integración aprobadas
Interfaz compilada y API en localhost:3001	Recorrido manual de venta y anulación comprobado

Proyecto académico de gestión comercial de relojes y audífonos inteligentes. Informe acompañado del código fuente, compilación, documentación, capturas y resultados de pruebas.

1. Propósito y alcance

Integrar los módulos de JXS Tech en una aplicación web funcional, conectando la interfaz de usuario con los servicios y la persistencia de datos. Se reutilizaron los avances de evidencias anteriores y se incorporó una interfaz integrada para los procesos comerciales.

Objetivos específicos

• Conectar autenticación, clientes, productos, inventario y ventas mediante una API.

• Aplicar validaciones, sesiones, desactivación lógica y transacciones para preservar la consistencia.

• Verificar el funcionamiento con pruebas unitarias, de integración y recorridos manuales.

• Documentar arquitectura, ambientes, instalación y ejecución de la versión compilada.

Productos y límites

Se entrega código JavaScript/JSX, CSS, archivos de configuración sin credenciales, interfaz compilada y documentación. El medio de pago se registra como dato. El sistema no procesa cobros bancarios, facturación electrónica, impuestos ni envíos.

La aplicación se ejecuta localmente en el puerto 3001. El código y su historial se consultan en el repositorio del proyecto: github.com/Sergiotech2026xl/JXSTech-GA8-AA1-EV01.

Guía de lectura

Las secciones siguientes presentan los requerimientos, la arquitectura y los cuatro diagramas, los mecanismos de seguridad, los ambientes y las pruebas. Al final se incluyen las cinco capturas suministradas por el aprendiz y un inventario de entrega.

2. Requerimientos e historias

ID	Historia de usuario	Criterio de aceptación
HU01	Como operador, quiero registrar una cuenta para acceder al sistema.	Acepta credenciales válidas, rechaza usuarios duplicados y almacena un hash de la contraseña.
HU02	Como operador, quiero iniciar y cerrar sesión para controlar mi acceso.	Credenciales válidas permiten entrar; el cierre invalida el token y las solicitudes sin sesión son rechazadas.
HU03	Como operador, quiero registrar y consultar clientes para utilizarlos en las ventas.	Guarda los seis campos obligatorios y permite consultarlos después de recargar. Documento y correo son únicos.
HU04	Como operador, quiero editar clientes para mantener su información actualizada.	Los cambios válidos se guardan y permanecen al recargar.
HU05	Como operador, quiero desactivar y reactivar clientes para controlar su disponibilidad.	Conserva sus datos y referencias históricas; un cliente inactivo no puede utilizarse en nuevas ventas.
HU06	Como operador, quiero registrar y consultar productos para mantener el catálogo.	Guarda SKU, nombre, categoría, marca, descripción opcional y precio. El SKU es único y el stock inicial es cero.
HU07	Como operador, quiero editar productos para actualizar el catálogo.	Guarda los datos válidos y conserva en las ventas anteriores los precios históricos.
HU08	Como operador, quiero desactivar y reactivar productos.	Solo permite desactivar productos con stock cero y bloquea nuevas ventas de productos inactivos.
HU09	Como operador, quiero registrar entradas y salidas para controlar las existencias.	Acepta cantidades enteras positivas y motivo obligatorio. Rechaza salidas superiores al stock.
HU10	Como operador, quiero consultar el historial para conocer los cambios de inventario.	Cada movimiento conserva producto, tipo, variación, motivo, operador, fecha y saldos.
HU11	Como operador, quiero registrar ventas para descontar los productos vendidos.	Requiere cliente activo, productos activos, cantidades válidas, stock suficiente y referencia única. El servidor calcula el total.
HU12	Como operador, quiero consultar el detalle de una venta para revisar su información histórica.	Muestra cliente, productos, cantidades, precios, total, fecha, medio de pago y estado.
HU13	Como operador, quiero anular una venta para restituir sus unidades.	Exige un motivo, conserva la venta y devuelve el stock una sola vez.
HU14	Como operador, quiero navegar entre los módulos para realizar mi trabajo.	El menú permite abrir Clientes, Productos, Inventario y Ventas, y cerrar sesión.

3. Arquitectura y tecnologías

Capa	Responsabilidad y elementos
Presentación	React y React DOM; formularios y tablas JSX; estilos CSS; pantallas Gestion*.jsx.
Comunicación	api.js centraliza Fetch, mensajes HTTP y envío del token Bearer.
Aplicación	Express organiza autenticación y rutas de clientes, productos, inventario y ventas. common.js centraliza validaciones.
Persistencia	database.js prepara MongoDB, índices y contexto de transacciones. Los módulos reciben sus dependencias mediante parámetros.

Herramientas y dependencias

Visual Studio Code se utilizó como editor e IDE de trabajo. Vite administra el ambiente de desarrollo y la compilación. ESLint revisa el frontend. Node.js aporta node:test y node:assert para las pruebas. El backend utiliza express, mongodb, bcryptjs y dotenv. Los archivos package-lock.json permiten reproducir las instalaciones con npm ci.

Decisiones de diseño y reutilización

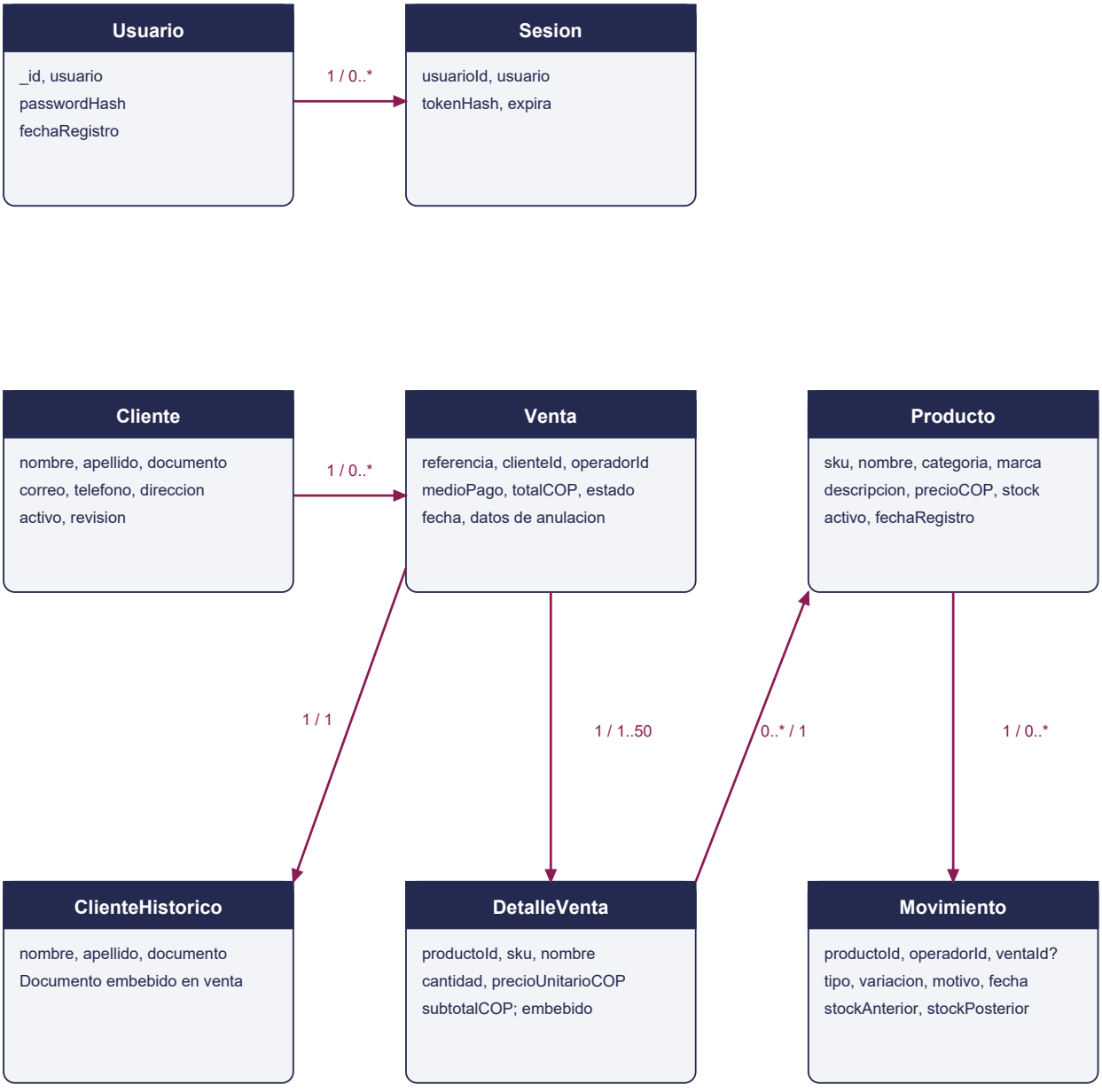
Se aplicaron composición de componentes React, cliente HTTP compartido, middleware de autenticación e inyección del contexto de persistencia. Ventas reutiliza mover() del módulo de inventario; ventas y anulaciones se procesan dentro de transacciones. Estas decisiones corresponden a la implementación; no se atribuye un patrón de repositorio ni una arquitectura de microservicios.

Metodología incremental

Se revisaron los avances, se creó una copia independiente y se desarrollaron incrementos de autenticación/clientes, productos, inventario y ventas. Cada incremento se comprobó manualmente y se registró en Git. Después se ejecutaron análisis estático, compilación y pruebas automáticas. No se declaran ceremonias de Scrum que no se realizaron.

4. Modelo conceptual del dominio

Las entidades representan documentos y datos embebidos; no clases JavaScript declaradas. La venta conserva una copia de los datos del cliente y de sus precios. Las relaciones adicionales indicadas al pie completan el modelo.

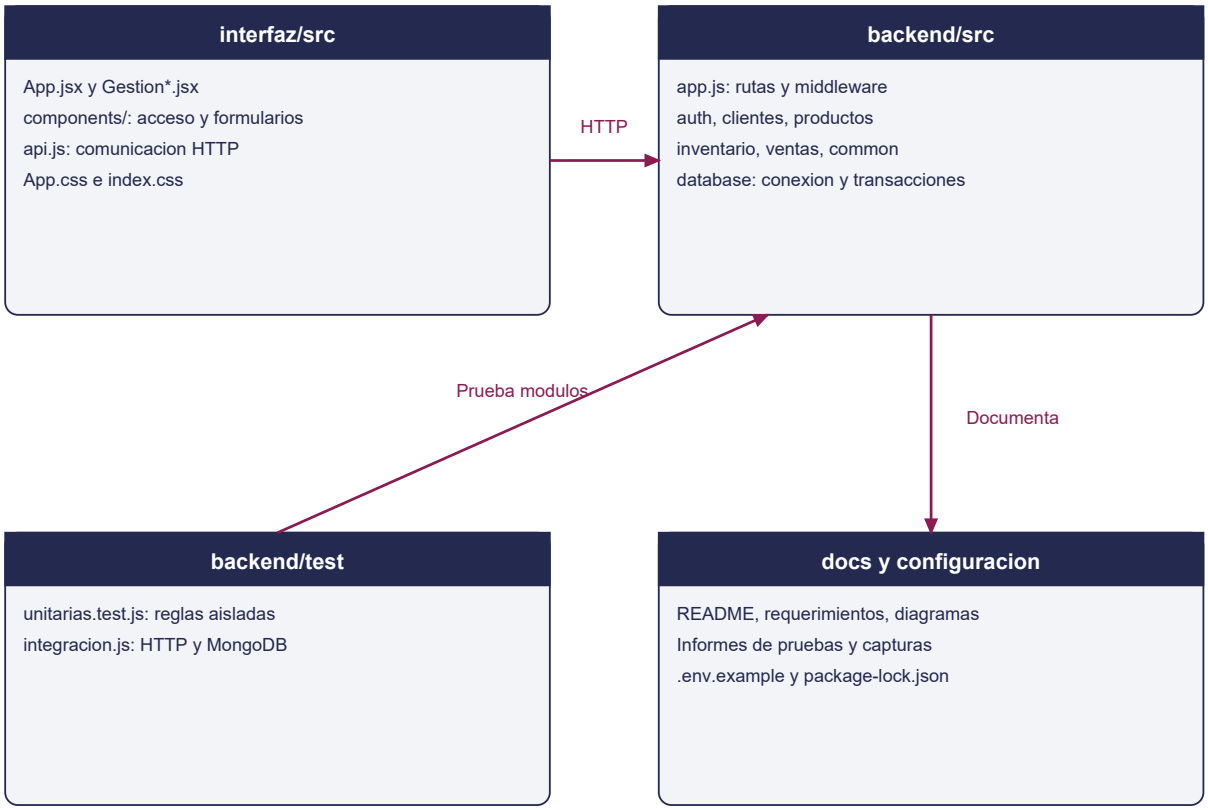


Relaciones adicionales: Usuario registra ventas y movimientos; una venta origina movimientos.

El detalle referencia al producto. ventaId es opcional en movimientos manuales.

5. Organizacion de paquetes

Los paquetes son agrupaciones logicas de carpetas. Las rutas reciben el contexto de persistencia; la suite unitaria sustituye colaboradores y la de integracion utiliza MongoDB real.

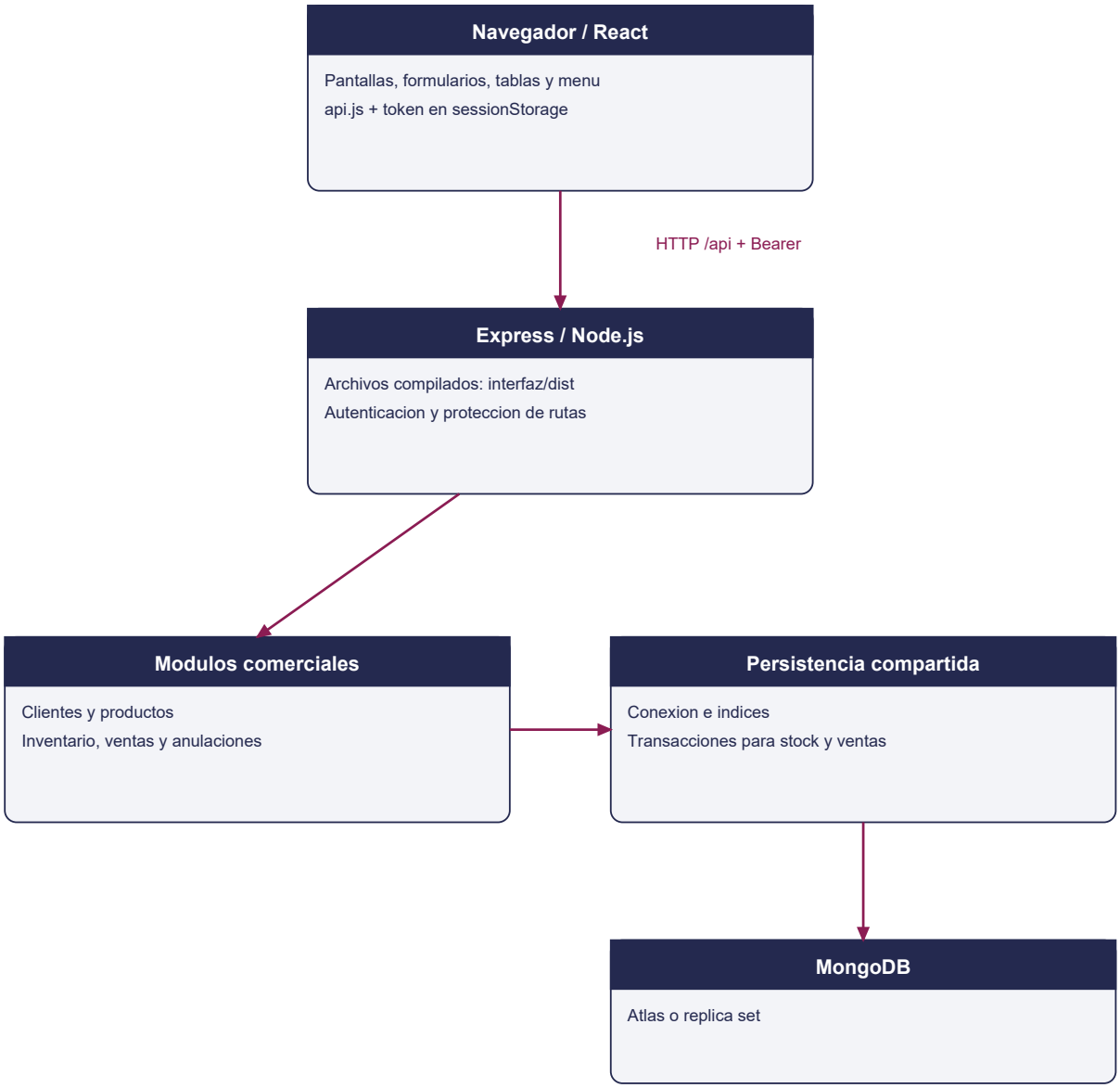


Dependencias internas: ventas reutiliza inventario; los modulos reutilizan common.

database crea el contexto que reciben los modulos por inyeccion de dependencias.

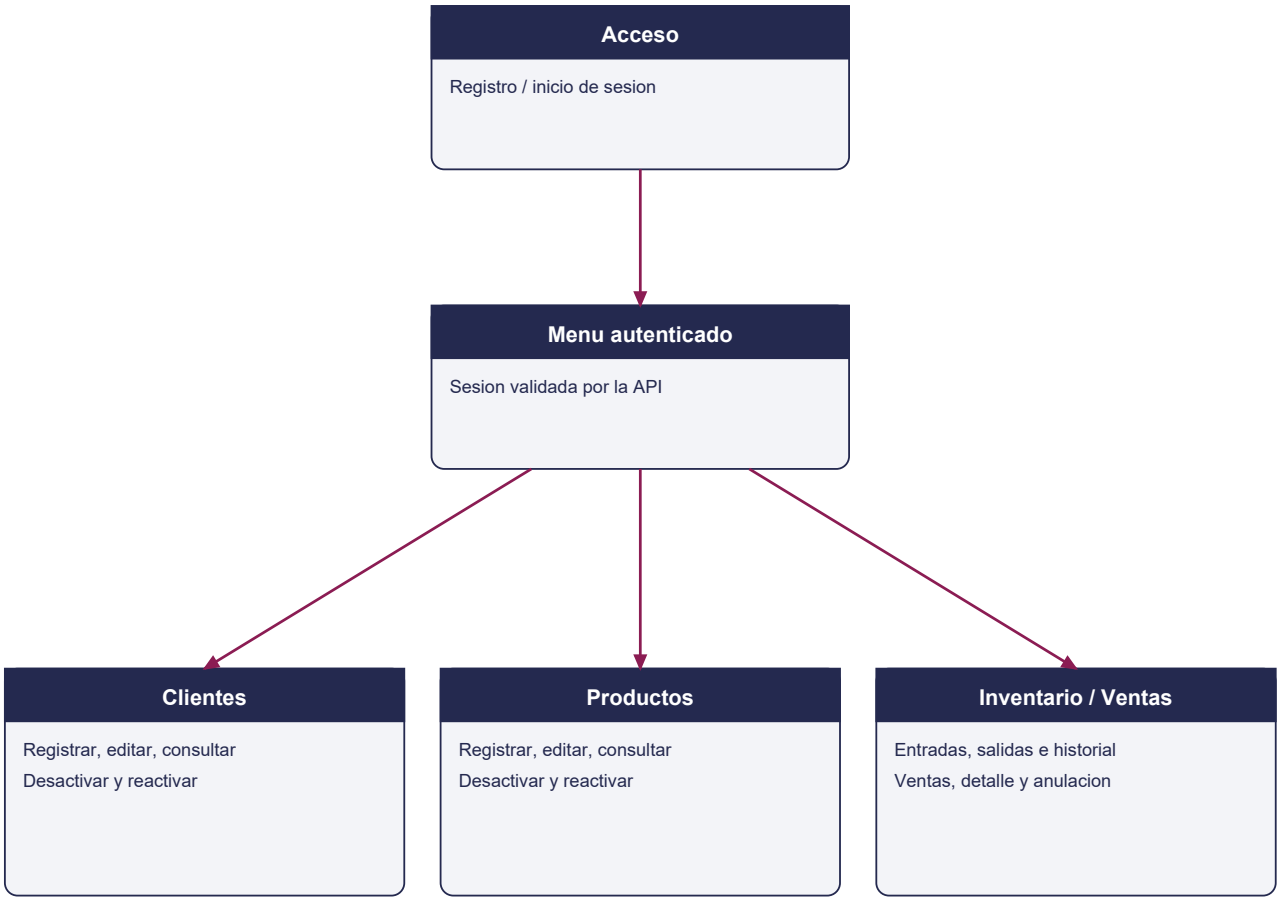
6. Componentes y capas

En ejecucion compilada, interfaz y API comparten el puerto 3001. En desarrollo, Vite usa 5173 y un proxy hacia 3001. Los modulos comerciales exigen una sesion valida.



7. Mapa de navegacion

El menu permite cambiar entre Clientes, Productos, Inventario y Ventas. La seleccion de pantalla se administra en el estado de React.



Cerrar sesion vuelve a Acceso desde cualquier modulo.

Al recargar, una sesion valida abre inicialmente Clientes.

8. Seguridad y reglas del negocio

Control	Implementación
Identidad	Contraseñas con hash bcrypt y usuario único. Tokens aleatorios; el servidor guarda el hash del token.
Sesión	Duración de 8 horas; consulta de sesión al recargar; logout invalida el token. Token de la pestaña en sessionStorage.
Acceso	Middleware exige Bearer token válido para clientes, productos, inventario y ventas. Todos los operadores registrados tienen iguales permisos.
Validación	Campos permitidos, formatos, longitudes, ObjectId y cantidades enteras; límite JSON de 32 KB y limitación de intentos de registro/login.
Privacidad de configuración	.env se excluye de Git y de la entrega. .env.example conserva solo marcadores de ejemplo.
Consistencia	Transacciones para registrar ventas, anularlas y modificar existencias; índices únicos evitan referencias y SKU repetidos.

Reglas comerciales comprobadas

• Los productos comienzan con stock cero; todo cambio genera un movimiento.

• No se admiten salidas superiores al stock ni desactivación de productos con existencias.

• Una venta requiere cliente y productos activos; su total se calcula en el servidor.

• El precio y los datos del cliente se conservan en la venta histórica.

• La anulación devuelve unidades una sola vez y conserva el registro original.

• Las bajas lógicas permiten reactivar clientes y productos sin romper referencias.

9. Instalación y ambientes

Entorno verificado: Windows, Node.js 24.21.0 y npm 11.19.0. Se requiere MongoDB Atlas o un replica set, permisos en la base y acceso de red habilitado. MongoDB standalone no satisface las transacciones utilizadas.

Preparación desde la carpeta JXSTech

```
cd backend
npm.cmd ci
Copy-Item .env.example .env
```

Editar localmente backend/.env y colocar una conexión propia. La entrega no incluye credenciales. La inicialización crea colecciones e índices; no hace falta importar los datos ficticios del recorrido manual.

```
PORT=3001
MONGODB_URI="CONEXION_PROPIA_DE_MONGODB"
DB_NAME=jxstech_ga8
```

Compilación y ejecución

```
cd ../interfaz
npm.cmd ci
npm.cmd run build
cd ../backend
npm.cmd start
```

Abrir http://localhost:3001/. Express sirve interfaz/dist y la API desde el mismo puerto. Se incluye la compilación ya generada. En una base nueva, crear un operador desde Registrar operador y luego iniciar sesión.

Ambiente de desarrollo y de pruebas

En desarrollo, ejecutar el backend y, en otra terminal dentro de interfaz, npm.cmd run dev. Vite usa localhost:5173 y dirige /api al puerto 3001. Las pruebas unitarias no necesitan base de datos. Las de integración crean una base temporal independiente y la eliminan al terminar; la cuenta requiere permisos para ambas operaciones.

10. Verificación y resultados

Verificación	Resultado y evidencia
Revisión estática	npm.cmd run lint: completó sin errores, según ejecución mostrada por el aprendiz.
Compilación	npm.cmd run build: generó interfaz/dist. Versión compilada comprobada en localhost:3001.
Pruebas unitarias	18 aprobadas, 0 fallidas; módulos de autenticación, clientes, productos, inventario, ventas y validaciones.
Pruebas de integración	38 aprobadas, 0 fallidas. Estado: aprobado. Base temporal eliminada: True.
Recorrido manual	Cliente y producto persistentes; entrada 10, salida 2, rechazo de salida 20; venta 2 unidades y anulación con devolución.

Reproducir comprobaciones

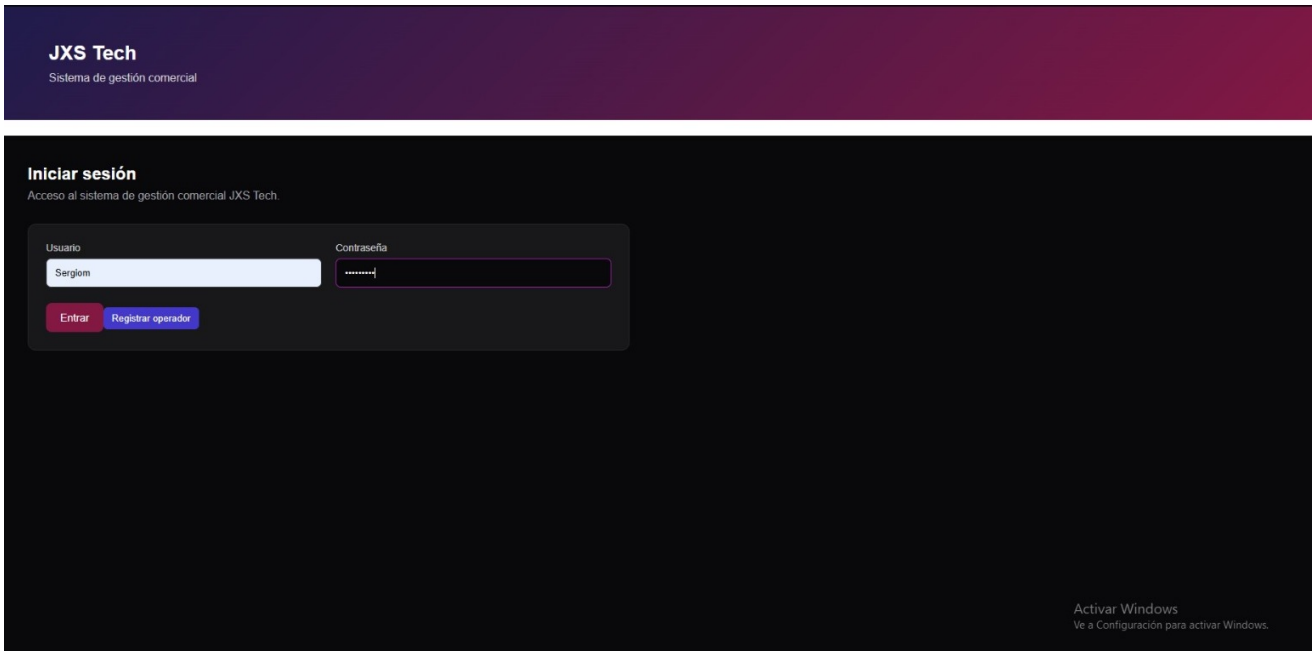
```
cd interfaz
npm.cmd run lint
npm.cmd run build
cd ../backend
npm.cmd run test:unitarias
npm.cmd test
```

Los informes originales se conservan en docs/resultados-unitarias-ga8.txt y docs/resultados-integracion-ga8.json. Las pruebas de integración incluyen acceso sin sesión, duplicados, stock insuficiente, reversión de ventas incompletas, concurrencia, conservación de precios históricos y restitución única en anulaciones.

La suite unitaria sustituye la persistencia por colaboradores controlados: verifica reglas aisladas, no durabilidad ni atomicidad real. Estas últimas se verifican mediante la suite de integración contra MongoDB.

11. Acceso de operadores

La interfaz permite registro e inicio de sesión. La contraseña se muestra enmascarada en la captura suministrada.



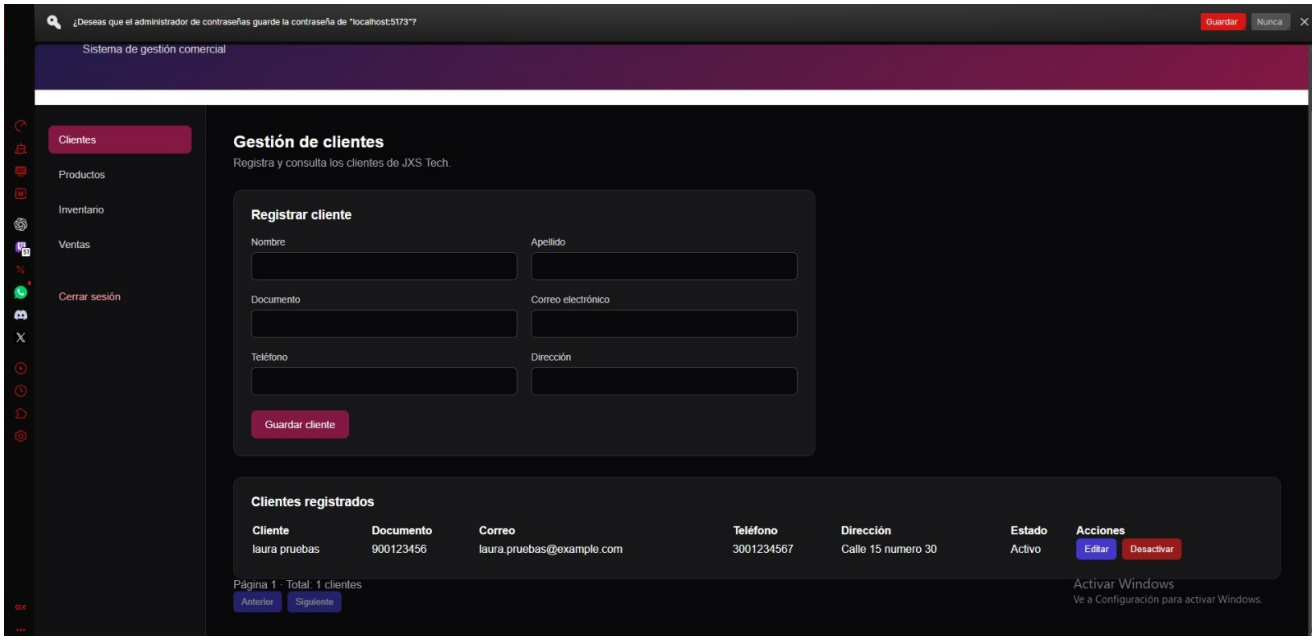
Fuente: captura de la aplicación suministrada por el aprendiz, 4 de noviembre de 2026. Se conserva la imagen original sin alterar sus resultados.

Lectura de la evidencia

Elemento	Dato visible
Pantalla	Iniciar sesión; botones Entrar y Registrar operador.
Credenciales	Usuario visible y contraseña enmascarada. No se incluyen contraseñas en el informe.

12. Gestión de clientes

Cliente ficticio Laura Pruebas registrado, activo y con dirección actualizada a Calle 15 numero 30.



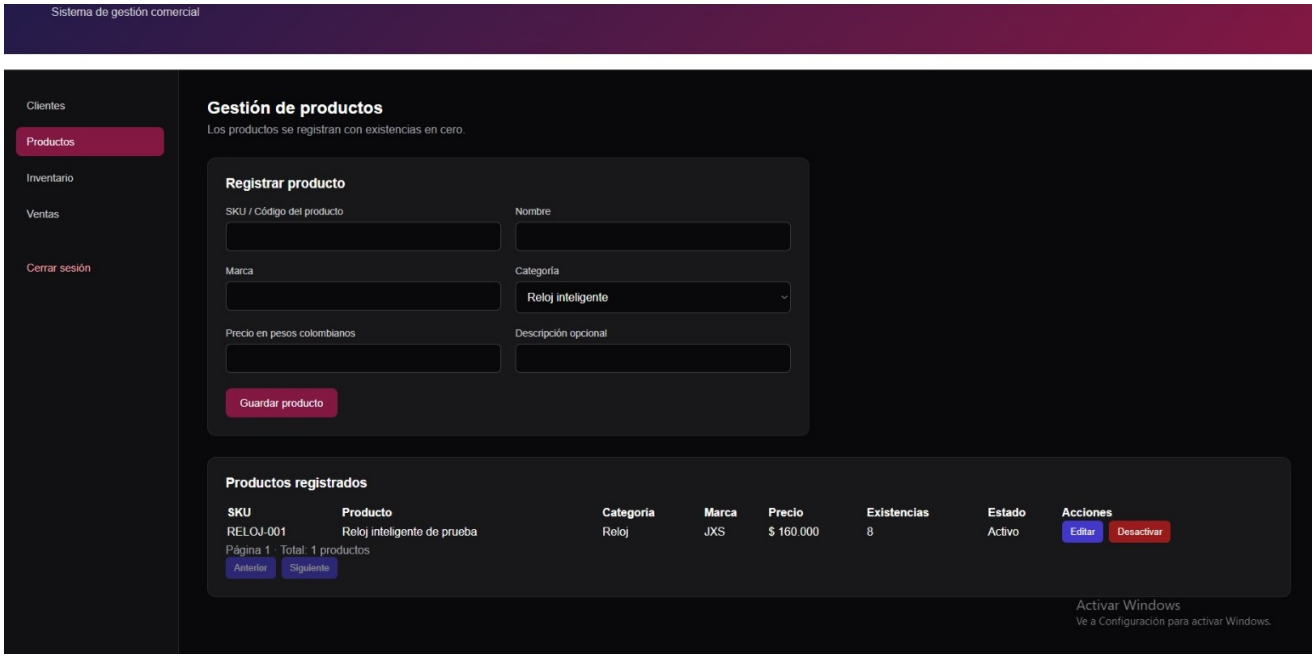
Fuente: captura de la aplicación suministrada por el aprendiz, 4 de noviembre de 2026. Se conserva la imagen original sin alterar sus resultados.

Lectura de la evidencia

Elemento	Dato visible
Cliente	Laura Pruebas; documento ficticio 900123456; estado Activo.
Actualización	Dirección: Calle 15 numero 30; correo laura.pruebas@example.com.

13. Catálogo de productos

RELOJ-001 activo, precio de 160.000 COP y saldo de 8 unidades después del recorrido comercial.



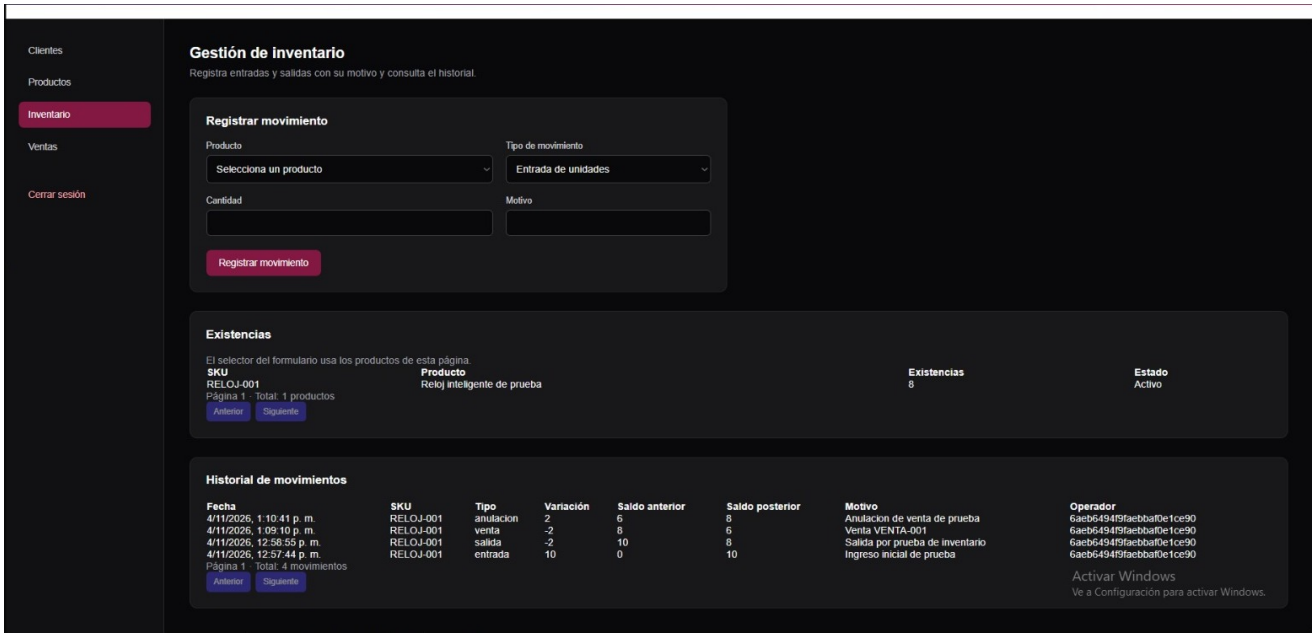
Fuente: captura de la aplicación suministrada por el aprendiz, 4 de noviembre de 2026. Se conserva la imagen original sin alterar sus resultados.

Lectura de la evidencia

Elemento	Dato visible
Producto	RELOJ-001; Reloj inteligente de prueba; marca JXS.
Resultado	Precio 160.000 COP; existencias 8; estado Activo.

14. Inventario e historial

Se conservan cuatro movimientos: entrada +10, salida -2, venta -2 y anulación +2. El saldo final es 8.



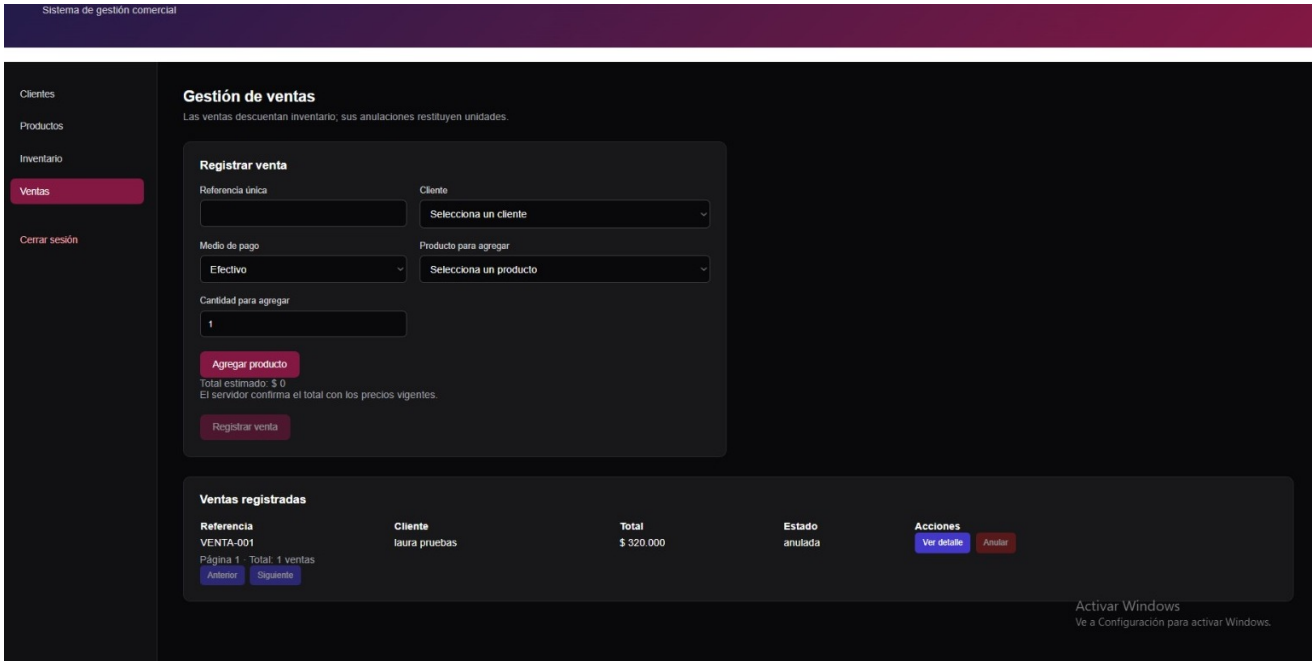
Fuente: captura de la aplicación suministrada por el aprendiz, 4 de noviembre de 2026. Se conserva la imagen original sin alterar sus resultados.

Lectura de la evidencia

Elemento	Dato visible
Entrada	Variación +10; saldo 0 a 10.
Salida manual	Variación -2; saldo 10 a 8.
Venta VENTA-001	Variación -2; saldo 8 a 6.
Anulación	Variación +2; saldo 6 a 8. Total: cuatro movimientos.

15. Venta anulada

VENTA-001 conserva el total histórico de 320.000 COP y estado anulada. El botón de anulación aparece deshabilitado.



Fuente: captura de la aplicación suministrada por el aprendiz, 4 de noviembre de 2026. Se conserva la imagen original sin alterar sus resultados.

Lectura de la evidencia

Elemento	Dato visible
Venta	VENTA-001; cliente Laura Pruebas.
Resultado	Total histórico 320.000 COP; estado anulada; botón Anular deshabilitado.

16. Entrega y control de versiones

El historial Git conserva los incrementos desarrollados manualmente. La carpeta de entrega incluye una copia independiente del proyecto y un bundle de Git para recuperar el historial sin depender de una publicación remota.

Elemento	Contenido
Informe	Este PDF y su copia dentro de docs del proyecto.
Proyecto	Código fuente, pruebas, dependencias declaradas, configuración de ejemplo y compilación interfaz/dist.
Diagramas	Fuentes Mermaid en docs/DIAGRAMAS.md y versiones SVG en docs/diagramas.
Evidencias	Capturas e informes originales de pruebas.
Historial	JXSTech_GA8.bundle y listado de commits.
Repositorio remoto	https://github.com/Sergiotech2026xl/JXSTech-GA8-AA1-EV01

Correspondencia con la evidencia

Los requerimientos e historias están en docs/REQUERIMIENTOS.md. La arquitectura, capas, dependencias, seguridad y metodología se explican en este informe y en el README. Se adjuntan los cuatro diagramas, pruebas unitarias por módulo, integración, configuración de base de datos e instrucciones de los ambientes. El código y la compilación acompañan el PDF.

Resultado

El recorrido verificado integra autenticación, clientes, productos, inventario y ventas. La anulación restituye las existencias y mantiene el historial. Los resultados consignados corresponden a evidencias observadas y a los informes de pruebas incluidos; no se atribuyen métricas de carga o disponibilidad no medidas.

Anexo A. Pruebas de integración

N.º	Caso	Resultado
1	Servidor y conexión disponibles	aprobada
2	Protección de todos los módulos sin sesión	aprobada
3	Registro y almacenamiento de hash de contraseña	aprobada
4	Usuario duplicado, contraseña corta y campos faltantes	aprobada
5	Login incorrecto y usuario inexistente devuelven 401	aprobada
6	Login correcto, sesión persistida y token guardado como hash	aprobada
7	Registro de cliente con los campos del proyecto previo	aprobada
8	Documento y correo de clientes únicos	aprobada
9	Correo e identificador inválidos y cliente inexistente	aprobada
10	Consulta individual, listado paginado y actualización de cliente	aprobada
11	Registro de reloj y audífonos con stock inicial cero	aprobada
12	Validación de SKU único, categoría, precio y stock no editable	aprobada
13	Listados y paginación inválida	aprobada
14	Entradas de inventario y registro de saldos	aprobada
15	Salida manual de inventario	aprobada
16	Stock insuficiente no crea movimiento ni altera saldo	aprobada
17	Cantidad cero, negativa, decimal y movimiento de venta manual rechazados	aprobada
18	Producto inexistente y límite de existencias	aprobada
19	Producto con existencias no se puede desactivar	aprobada
20	Venta calcula precios y total en servidor y descuenta stock	aprobada
21	Referencia duplicada evita registrar o descontar dos veces	aprobada
22	Venta con segunda línea sin stock revierte toda la operación	aprobada
23	Venta rechaza items vacíos, repetidos y precio manipulado	aprobada
24	Venta rechaza cliente o producto inexistente y no deja descuentos	aprobada
25	Historial conserva precio original al actualizar catálogo	aprobada
26	Anulación restituye cantidades y conserva la venta	aprobada

N.º	Caso	Resultado
27	Anulación repetida no devuelve stock dos veces	aprobada
28	Historial de movimientos filtrado por producto	aprobada
29	Ventas concurrentes: solo una compra la última existencia	aprobada
30	Referencia concurrente conserva una sola venta y descuento	aprobada
31	Anulaciones concurrentes: una sola restitución	aprobada
32	Baja lógica de producto conserva historial y bloquea nuevas ventas	aprobada
33	Anular una venta de producto inactivo también restituye stock	aprobada
34	Baja lógica de cliente preserva datos y bloquea ventas	aprobada
35	Reactivación de cliente y producto recupera la operación comercial	aprobada
36	Errores JSON, tamaño máximo, ruta y secreto no expuesto	aprobada
37	Sesión expirada es rechazada aunque TTL aún no elimine el registro	aprobada
38	Logout invalida el token en todas las rutas protegidas	aprobada

Anexo B. Pruebas unitarias

N.º	Caso	Resultado
1	Común: limpia espacios y rechaza texto demasiado corto	Aprobada
2	Común: rechaza cantidades decimales y fuera del rango	Aprobada
3	Autenticación: registra el usuario con contraseña cifrada mediante hash	Aprobada
4	Autenticación: rechaza una contraseña corta	Aprobada
5	Autenticación: bloquea solicitudes sin token	Aprobada
6	Clientes: normaliza datos y registra un cliente activo	Aprobada
7	Clientes: rechaza un correo inválido	Aprobada
8	Clientes: rechaza campos que no pertenecen al contrato	Aprobada
9	Productos: normaliza SKU e inicia existencias en cero	Aprobada
10	Productos: rechaza una categoría desconocida	Aprobada
11	Productos: rechaza precios con decimales	Aprobada
12	Inventario: una entrada genera el movimiento con los saldos correctos	Aprobada
13	Inventario: una salida insuficiente no registra un movimiento	Aprobada
14	Inventario: rechaza movimientos de cantidad cero	Aprobada
15	Ventas: rechaza una venta sin productos	Aprobada
16	Ventas: rechaza productos repetidos en distintas líneas	Aprobada
17	Ventas: calcula el total usando el precio del catálogo	Aprobada
18	Ventas: impide volver a anular una venta ya anulada	Aprobada